TP02

Master SIS - Modélisation géométrique
Courbes de Bézier

EXERCICE 1

Afin d'obtenir les points d'échantillonnage d'une courbe de Bézier, il est nécessaire de calculer les différents coefficients qui composent la courbe. Écrivez alors la fonction **`computeBezierCoeff`** qui renvoie la valeur du polynôme de Bernstein B_i^n au paramètre **`t`**. Vous utiliserez la formule :

$$B_i^n(t) = C_n^i t^i (1-t)^{n-i} \quad \text{où } C_n^i = \frac{n!}{i!(n-i)!} \text{ et } t \in [0,1]$$

Pour obtenir le coefficient C_n^i, utilisez soit un produit factoriel, soit la formule du triangle de Pascal.

```
double computeBezierCoeff(int i, int n, double t)
```

La seconde étape est le calcul d'un point d'échantillonnage. Développez la fonction **`computeBezierPoint`** qui permet d'obtenir la position d'un point d'échantillonnage **`q`** au paramètre **`t`** en fonction des points de contrôle et du degré de la courbe :

$$q = \sum_{i=0}^{n} B_i^n(t) P_{i_0+i}$$

`i0` correspond en fait à l'indice du premier point de contrôle utilisé pour le calcul de **`q`**. Ce paramètre sera utile pour définir une courbe de Bézier à partir de plusieurs segments.

```
void computeBezierPoint(bezierCurve * curve, int i0, double t, mlVec3 q)
```

Étendez le calcul, dans la fonction **`computeBezierCurve`**, de manière à calculer l'intégralité des points d'échantillonnage d'une courbe de Bézier.

```
void computeBezierCurve(bezierCurve * curve)
```

EXERCICE 2

Écrivez la procédure **`drawBezierCurve`** pour réaliser l'affichage d'une courbe de Bézier calculée par les fonctions précédemment développées.

```
void drawBezierCurve(bezierCurve * curve)
```

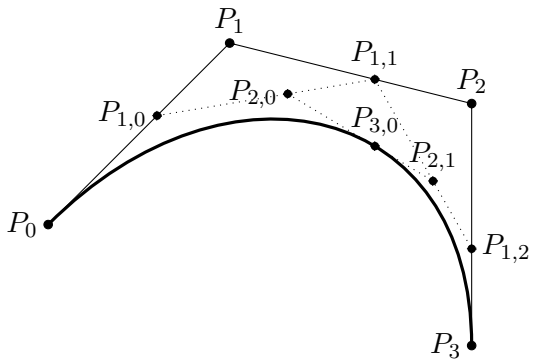
Ajoutez ensuite à votre programme la possibilité de modifier interactivement le degré ainsi que la résolution de la courbe de Bézier.

EXERCICE 3

Modifiez le programme pour afficher la construction de De Casteljau au paramètre **`t`**.

```
void computeBezierDeCasteljau(bezierCurve * curve, double t)
```

La valeur de `t` est initialement fixée à 0.8. Celle-ci doit être également modifiable interactivement. Pour la construction de De Casteljau, vous utiliserez l'algorithme rappelé par la figure suivante :



Points de contrôle	P_0	P_1	P_2	P_3
Construction	P_0	P_1	P_2	P_3
	$P_{1,0}$	$P_{1,1}$	$P_{1,2}$	$\times$
	$P_{2,0}$	$P_{2,1}$	$\times$	$\times$
	$P_{3,0}$	$\times$	$\times$	$\times$

$$P_{j+1,i} = (1-t)P_{j,i} + tP_{j,i+1}$$

Bonus

Ajoutez à votre programme la possibilité de transformer des courbes hermitiennes cubiques en courbes de Bézier cubiques. Une courbe de Bézier ainsi créée doit conserver le même aspect visuel que la courbe d'Hermite qui a été utilisée pour la transformation.